

Distributed Systems

Lecture9: Threads

Mohammad Reza Shakournia

shakournia@ut.ac.ir

School of Electrical and Computer Engineering,
University of Tehran,
Tehran, Iran

Introduction to Threads

- We build virtual processors in software, on top of physical processors:
  - **Processor:** Provides a set of instructions along with the capability of automatically executing a series of those instructions.
  - **Thread:** A minimal software processor in whose context a series of instructions can be executed. Saving a thread context implies stopping the current execution and saving all the data needed to continue the execution at a later stage.
  - **Process:** A software processor in whose context one or more threads may be executed. Executing a thread, means executing a series of instructions in the context of that thread.

Context Switching

- We have three contexts defined as below:
  - **Processor context:** The minimal collection of values stored in the registers of a processor used for the execution of a series of instructions (e.g., stack pointer, addressing registers, program counter).
  - **Thread context:** The minimal collection of values stored in registers and memory, used for the execution of a series of instructions (i.e., processor context, state).
  - **Process context:** The minimal collection of values stored in registers and memory, used for the execution of a thread (i.e., thread context, but now also at least MMU register values).

Context Switching

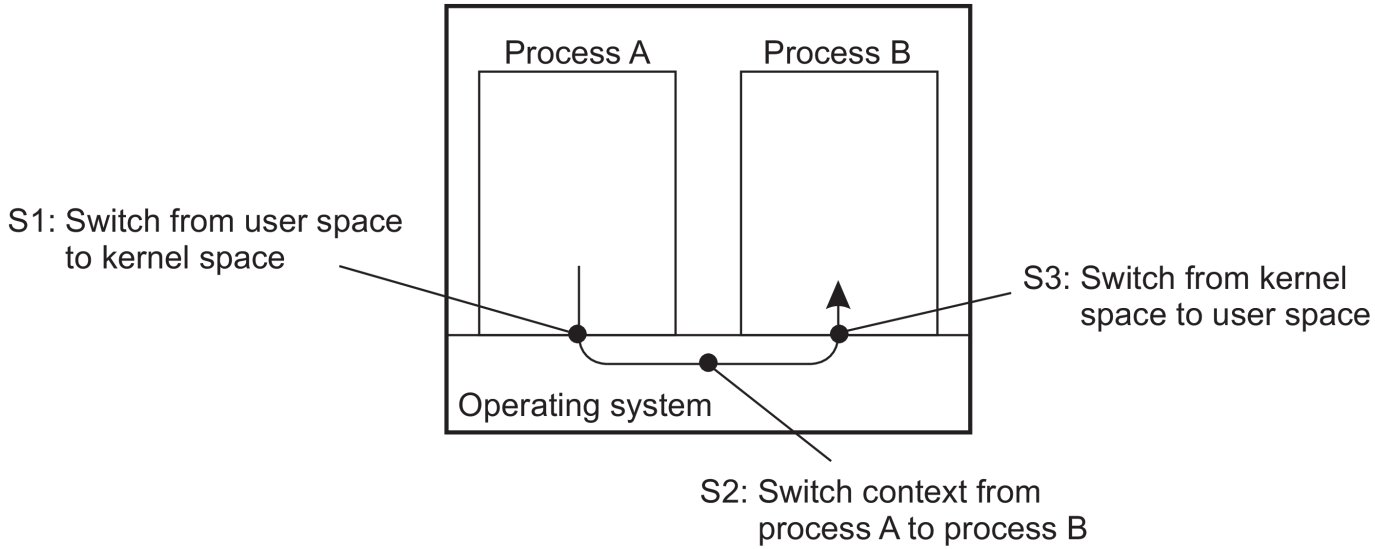
- Observations:
  1. Threads share the same address space. Thread context switching can be done entirely independent of the operating system.
  2. Process switching is generally (somewhat) more expensive as it involves getting the OS in the loop, i.e., trapping to the kernel.
  3. Creating and destroying threads is much cheaper than doing so for processes.

Why Use Threads

- **Avoid needless blocking:** a single-threaded process will block when doing I/O; in a multithreaded process, the operating system can switch the CPU to another thread in that process.
- **Exploit parallelism:** the threads in a multithreaded process can be scheduled to run in parallel on a multiprocessor or multicore processor.
- **Avoid process switching:** structure large applications not as a collection of processes, but through multiple threads.

Avoid Process Switching

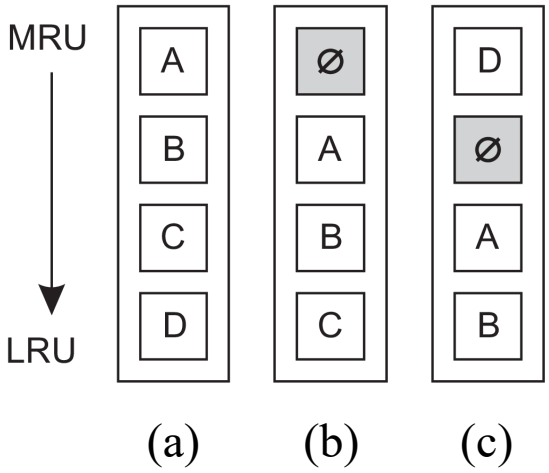
- Avoid expensive context switching:



- Trade-offs:
  - Threads use the same address space: more prone to errors
  - No support from OS/HW to protect threads using each other's memory
  - Thread context switching may be faster than process context switching

The Cost of a Context Switch

- Consider a simple clock-interrupt handler:
  - **Direct Costs:** actual switch and executing code of the handler
  - **Indirect Costs:** other costs, notably caused by messing up the cache
- What a context switch may cause: indirect costs:



(a) Before the context switch
(b) After the context switch
(c) After accessing block D.

A Simple Example in Python

```
1  from multiprocessing import Process
2  from time import *
3  from random import *
4
5  def sleeper(name):
6      t = gmtime()
7      s = randint(1,20)
8      txt = str(t.tm_min)+':'+str(t.tm_sec)+' '+name+' is going to sleep for '+str(s)+' seco
9      print(txt)
10     sleep(s)
11     t = gmtime()
12     txt = str(t.tm_min)+':'+str(t.tm_sec)+' '+name+' has woken up'
13     print(txt)
14
15 if __name__ == '__main__':
16     p = Process(target=sleeper, args=('eve',))
17     q = Process(target=sleeper, args=('bob',))
18     p.start(); q.start()
19     p.join(); q.join()
```

```
40:23 eve is going to sleep for 14 seconds
40:23 bob is going to sleep for 4 seconds
40:27 bob has woken up
40:37 eve has woken up
```

A Simple Example in Python

```
1  from multiprocessing import Process
2  from threading import Thread
3
4  shared_x = random.randint(10,99)
5
6  def sleeping(name):
7      global shared_x
8      s = randint(1,20)
9      sleep(s)
10     shared_x = shared_x + 1
11
12 def sleeper(name):
13     sleeplist = list()
14     for i in range(3):
15         subsleeper = Thread(target=sleeping, args=(name+' '+str(i),))
16         sleeplist.append(subsleeper)
17
18     for s in sleeplist: s.start()
19     for s in sleeplist: s.join()
```

A Simple Example in Python

```
eve sees shared x being 71
53:21 eve 0 is going to sleep for 20 seconds
bob sees shared x being 84
53:21 eve 1 is going to sleep for 15 seconds
53:21 eve 2 is going to sleep for 3 seconds
53:21 bob 0 is going to sleep for 8 seconds
53:21 bob 1 is going to sleep for 16 seconds
53:21 bob 2 is going to sleep for 8 seconds
53:24 eve 2 has woken up, seeing shared x being 72
53:29 bob 0 has woken up, seeing shared x being 85
53:29 bob 2 has woken up, seeing shared x being 86
53:36 eve 1 has woken up, seeing shared x being 73
53:37 bob 1 has woken up, seeing shared x being 87
bob sees shared x being 87
53:41 eve 0 has woken up, seeing shared x being 74
eve sees shared x being 74
```

Threads and Operating Systems

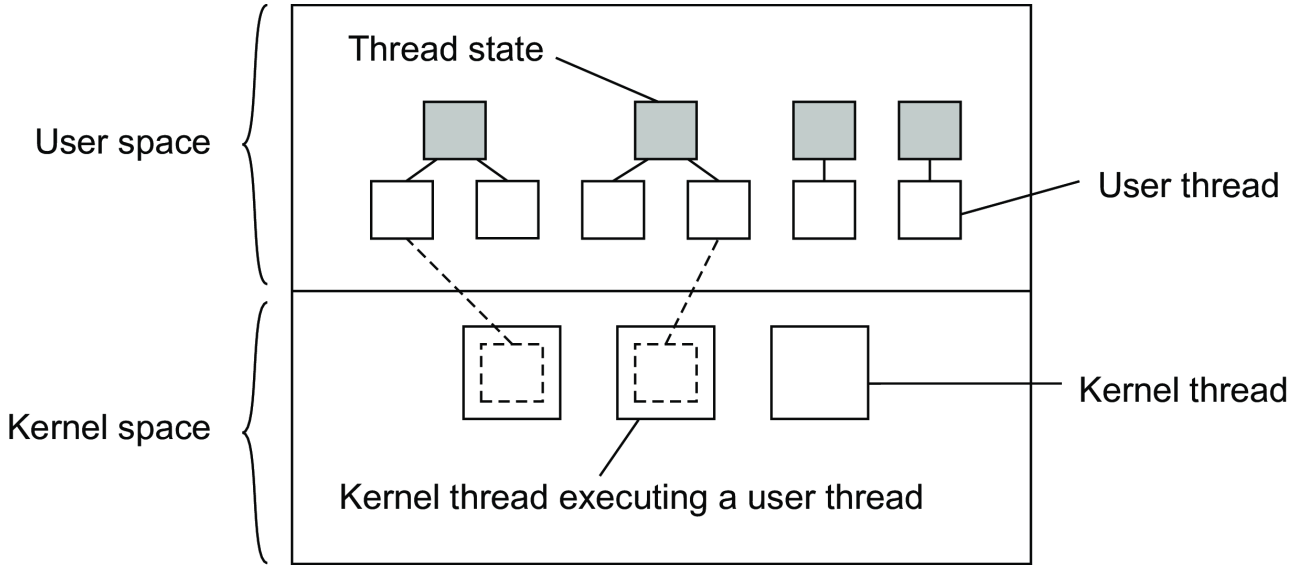
- **Should an OS kernel provide threads, or should they be implemented as user-level packages?**
- User-space solution:
  - All operations can be completely handled within a single process ⇒ implementations can be extremely efficient.
  - All services provided by the kernel are done on behalf of the process in which a thread resides ⇒ if the kernel decides to block a thread, the entire process will be blocked.
  - Threads are used when there are many external events: threads block on a per-event basis ⇒ if the kernel can't distinguish threads, how can it support signaling events to them?

Threads and Operating Systems

- **Kernel solution:** The whole idea is to have the kernel contain the implementation of a thread package. This means that all operations return as system calls:
  - **Operations that block a thread are no longer a problem:** the kernel schedules another available thread within the same process.
  - **Handling external events is simple:** the kernel (which catches all events) schedules the thread associated with the event.
  - The problem is (or used to be) the loss of efficiency because each thread operation requires a trap to the kernel.
- Try to mix user-level and kernel-level threads into a single concept, however, performance gain has not turned out to generally outweigh the increased complexity.

Combining User-level And Kernel-level Threads

- Introduce a two-level threading approach: kernel threads that can execute user-level threads.



User and Kernel Threads Combined

- Principle operation are as below:
  - User thread does system call ⇒ the kernel thread that is executing that user thread, blocks. The user thread remains bound to the kernel thread.
  - The kernel can schedule another kernel thread having a runnable user thread bound to it. Note: this user thread can switch to any other runnable user thread currently in user space.
  - A user thread calls a blocking user-level operation ⇒ do context switch to a runnable user thread, (then bound to the same kernel thread).
  - When there are no user threads to schedule, a kernel thread may remain idle, and may even be removed (destroyed) by the kernel.

Using Threads at the Client Side

- **Multithreaded web client - Hiding network latencies:**
  - Web browser scans an incoming HTML page, and finds that more files need to be fetched.
  - Each file is fetched by a separate thread, each doing a (blocking) HTTP request.
  - As files come in, the browser displays them.
- **Multiple request-response calls to other machines (RPC):**
  - A client does several calls at the same time, each one by a different thread.
  - It then waits until all results have been returned.
  - Note: if calls are to different servers, we may have a linear speed-up.

Multithreaded Clients: Does it help?

- **Thread-level parallelism: TLP**
  - Let c_i denote the fraction of time that exactly i threads are being executed simultaneously.

$$TLP = \frac{\sum_{i=1}^{N} i.c_i}{1 - c_0}$$

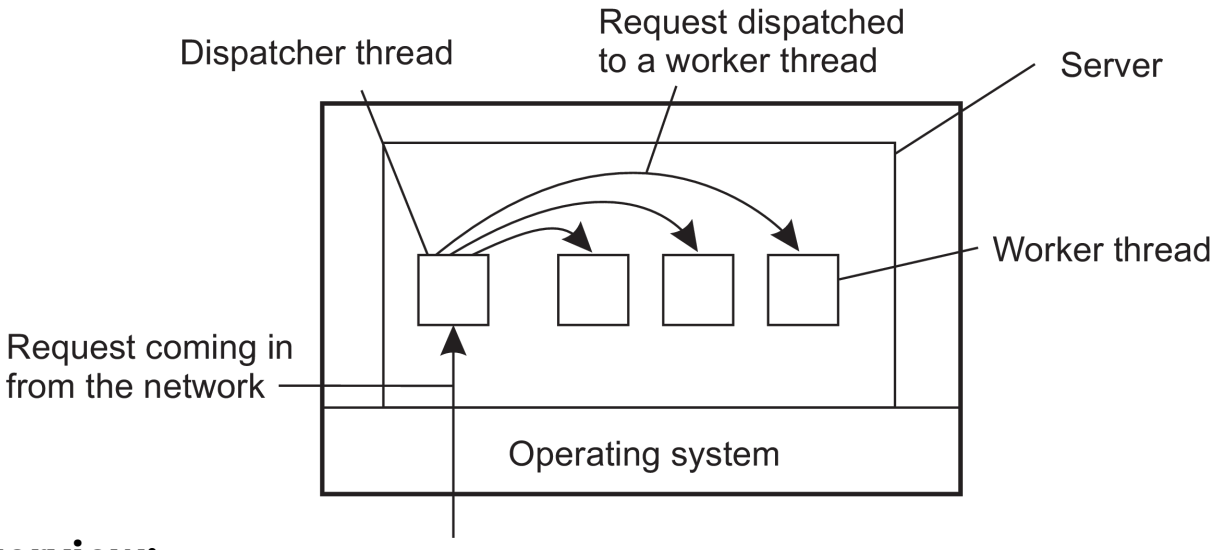
  - with N the maximum number of threads that (can) execute at the same time.
- **Practical measurements:**
  - A typical Web browser has a TLP value between 1.5 and 2.5 ⇒ threads are primarily used for logically organizing browsers.

Using Threads at the Server Side

- **Improve performance:**
  - Starting a thread is cheaper than starting a new process.
  - Having a single-threaded server prohibits simple scale-up to a multiprocessor system.
  - As with clients: hide network latency by reacting to next request while previous one is being replied.
- **Better structure:**
  - Most servers have high I/O demands. Using simple, well-understood blocking calls simplifies the structure.
  - Multithreaded programs tend to be smaller and easier to understand due to simplified flow of control.

Why Multithreading is Popular: Organization

- **Dispatcher/Worker Model:**



- **Overview:**

Model	Characteristics
Multithreading	Parallelism, blocking system calls
Single-threaded process	No parallelism, blocking system calls
Finite-state machine	Parallelism, nonblocking system calls

Wrap-Up + What's Next

In the next lecture, we will discuss virtualization including:

- Principle of virtualization,
- Container,
- Comparison between VM and Container,
- Application of VMs to Distributed Systems.